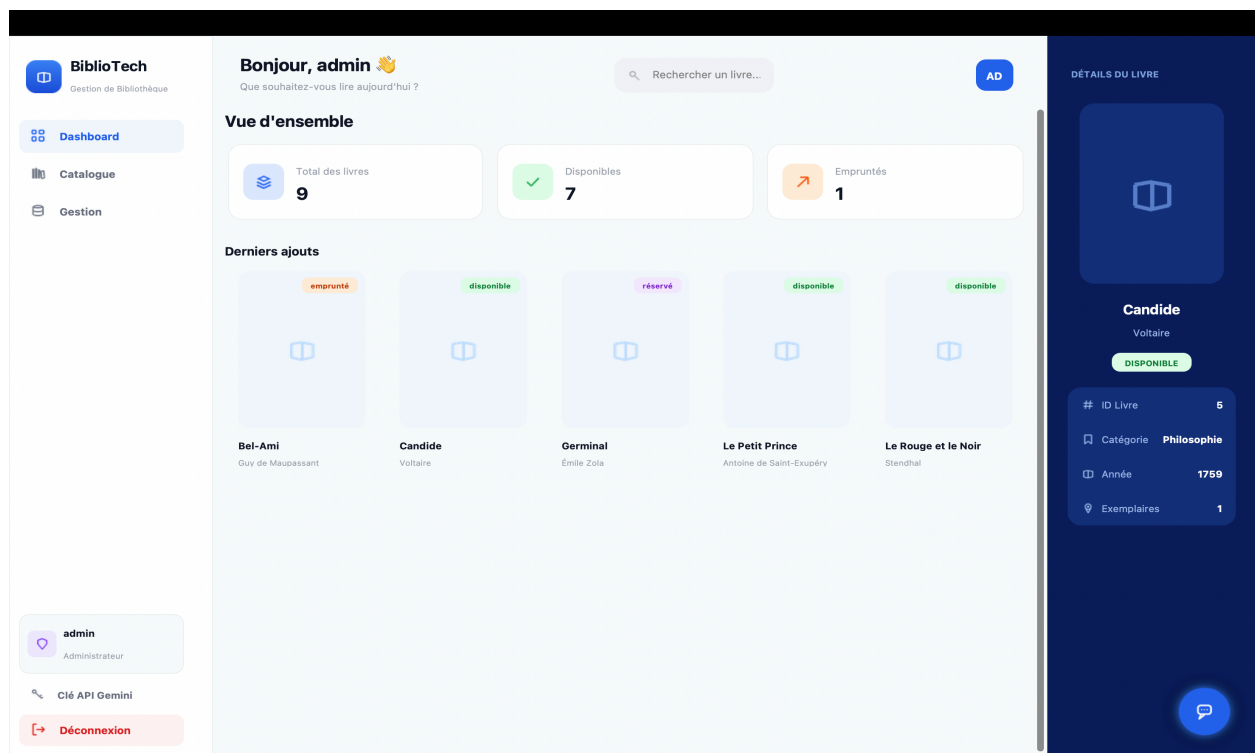


BiblioTech

Bibliothèque Intelligente avec Chatbot IA
Rapport de Projet Semestriel — Python

Réalisé par Hiba Dhaoui

Année universitaire 2025 – 2026



Sommaire

1. Introduction et objectifs.....	3
2. Choix technologiques.....	3
2.1 Choix de SQLite.....	3
2.2 Choix de Google Gemini.....	4
3. Architecture générale.....	4
4. Conception de la base de données.....	4
5. Implémentation du CRUD.....	5
6. Authentification et gestion des rôles.....	5
7. Fonctionnement du chatbot intelligent.....	6
Étape 1 — Extraction des données.....	6
Étape 2 — Création du contexte.....	6
Étape 3 — Construction du prompt.....	6
Étape 4 — Génération de la réponse.....	6
8. Interface graphique.....	6
9. Sécurité et bonnes pratiques.....	6
10. Difficultés rencontrées.....	7
11. Perspectives d'amélioration.....	7
12. Conclusion.....	7

1. Introduction et objectifs

BiblioTech est une application Python de gestion de bibliothèque combinant un système complet de gestion des livres et un assistant conversationnel basé sur l'intelligence artificielle.

Le projet répond à deux fonctionnalités principales définies dans le cahier des charges : la gestion CRUD des livres (ajouter, afficher, modifier, supprimer, rechercher) et un chatbot intelligent capable de répondre aux questions des utilisateurs en langage naturel à partir des données réelles de la bibliothèque.

Au-delà de ces exigences, l'application propose une interface graphique moderne, un système d'authentification avec rôles, la gestion des couvertures de livres et une journalisation des actions administratives.

2. Choix technologiques

Le projet a été développé entièrement en Python. Les technologies suivantes ont été retenues après analyse des différentes options proposées.

Couche	Technologie	Justification
Langage	Python 3.13	Imposé par le cahier des charges
Interface	CustomTkinter	Rendu visuel moderne (coins arrondis, thèmes)
Base de données	SQLite (sqlite3)	Intégrée à Python, fichier unique, sans serveur
Intelligence artificielle	Google Gemini	Clé gratuite, excellent langage naturel
Images	Pillow (PIL)	Icônes vectorielles et couvertures

2.1 Choix de SQLite

SQLite est un moteur de base de données relationnelle léger fonctionnant à partir d'un fichier unique. Contrairement à MySQL ou PostgreSQL, il ne nécessite aucun serveur ni configuration complexe.

Il est intégré nativement à Python via le module sqlite3, ce qui simplifie le développement. Ses performances sont largement suffisantes pour une application locale, et sa portabilité est idéale : toutes les données tiennent dans un seul fichier facilement sauvegardé ou transféré.

La base contient quatre tables : livres, utilisateurs, clés API et journal d'activité.

2.2 Choix de Google Gemini

Pour le chatbot, plusieurs solutions ont été étudiées (OpenAI GPT, Anthropic Claude, Ollama, Google Gemini). Le choix s'est porté sur Gemini pour sa version gratuite adaptée aux projets académiques et ses excellentes capacités de compréhension du langage naturel.

Contrairement à une recherche par mots-clés, Gemini interprète différentes formulations d'une même question. Les phrases « Quels livres de philosophie possédez-vous ? » et « Je cherche un ouvrage philosophique disponible » sont comprises de façon équivalente. Le modèle utilisé est Gemini Flash, qui offre le meilleur compromis entre rapidité, coût et qualité.

3. Architecture générale

Une architecture en couches a été adoptée afin de séparer clairement les responsabilités et de faciliter la maintenance. Une modification peut être apportée à une couche sans impacter l'ensemble du système.

Couche	Rôle
Interface utilisateur	Affiche les données et transmet les actions aux services
Services	Cœur fonctionnel : validation et logique métier
Modèles	Objets métiers (classe Book)
Base de données	Seul composant communiquant avec SQLite
Chatbot IA	Interactions avec Gemini

Cette séparation garantit que l'interface ne réalise aucun traitement métier complexe, et que toutes les requêtes SQL sont centralisées dans une seule couche, limitant les risques d'erreurs.

4. Conception de la base de données

La base est conçue selon les principes des bases relationnelles afin d'éviter les redondances et de garantir l'intégrité des informations. Chaque livre est identifié de manière unique par un identifiant numérique.

La table « livres » contient : identifiant, titre, auteur, catégorie, année de publication, quantité disponible, statut et chemin de la couverture. La table « utilisateurs » contient l'identifiant, le

nom d'utilisateur, le mot de passe haché et le rôle. Une table stocke la clé API Gemini, et une table de journalisation enregistre les actions des administrateurs.

L'organisation retenue reste légère tout en permettant d'ajouter facilement de futures fonctionnalités telles que la gestion des emprunts, des réservations ou des notifications.

5. Implémentation du CRUD

Le CRUD (Create, Read, Update, Delete) constitue la fonctionnalité principale du système. Chaque opération suit le même circuit : l'interface transmet la demande au service, qui valide les données avant de solliciter la base.

Opération	Description
Ajouter	Formulaire validé puis enregistrement, avec couverture optionnelle
Consulter	Récupération dynamique et affichage en cartes modernes
Modifier	Formulaire pré-rempli puis remplacement des données
Supprimer	Confirmation obligatoire avant suppression définitive
Rechercher	Filtres par titre, auteur, catégorie et statut

La recherche filtre les résultats instantanément, améliorant la rapidité de consultation du catalogue.

6. Authentification et gestion des rôles

Afin de protéger l'accès aux fonctionnalités sensibles, un système d'authentification a été mis en place. Le mot de passe fourni n'est jamais enregistré en clair : il est transformé via l'algorithme SHA-256, qui produit une empreinte numérique unique. Même en cas d'accès à la base, il est impossible de retrouver les mots de passe originaux.

Deux rôles sont gérés. L'administrateur possède tous les droits :

- Gérer les livres (ajout, modification, suppression)
- Consulter les statistiques
- Configurer le chatbot (clé API)
- Consulter les journaux d'activité

Le lecteur dispose uniquement d'un accès en consultation, ce qui évite toute modification non autorisée.

7. Fonctionnement du chatbot intelligent

Le chatbot est la fonctionnalité la plus innovante du projet. Plutôt que de naviguer dans des menus, l'utilisateur pose simplement une question. Le fonctionnement repose sur l'approche Retrieval Augmented Generation (RAG), qui enrichit les requêtes envoyées à l'IA avec les données réelles du système.

Étape 1 — Extraction des données

Le système récupère depuis SQLite les titres, auteurs, catégories et disponibilités.

Étape 2 — Création du contexte

Les informations sont transformées en texte structuré compréhensible par Gemini.

Étape 3 — Construction du prompt

Le contexte est associé à la question de l'utilisateur pour former le prompt final.

Étape 4 — Génération de la réponse

Gemini analyse le contexte et génère une réponse en langage naturel, affichée dans l'interface. Le chatbot reste ainsi toujours connecté aux données réelles du catalogue. Un mode hors-ligne de secours répond depuis SQLite si l'API est indisponible.

8. Interface graphique

L'interface a été conçue selon des principes de design modernes : navigation intuitive, affichage clair, cohérence graphique et simplicité d'utilisation. L'application comprend un écran de connexion, un tableau de bord, un catalogue, un espace de gestion et un chatbot intégré.

Les icônes sont dessinées vectoriellement avec Pillow plutôt qu'avec des emojis, garantissant un rendu minimaliste et cohérent. La palette, inspirée des tableaux de bord professionnels, repose sur un bleu d'accent et des gris clairs.

9. Sécurité et bonnes pratiques

Plusieurs mécanismes de sécurité ont été implémentés :

- Requêtes SQL paramétrées pour prévenir les injections SQL

- Hachage des mots de passe par SHA-256
- Clé API Gemini jamais intégrée en dur dans le code (base ou variable d'environnement)
- Gestion des exceptions pour éviter les interruptions brutales

Ces mesures contribuent à rendre l'application robuste et fiable. La base de données contenant la clé est exclue du dépôt git afin de ne jamais être publiée.

10. Difficultés rencontrées

La principale difficulté concernait l'intégration du chatbot avec les données réelles : il a fallu concevoir un mécanisme extrayant automatiquement les informations pertinentes avant chaque requête.

La gestion des appels réseau constituait également un défi, car les requêtes vers Gemini pouvaient bloquer l'interface. L'utilisation de threads a résolu ce problème. Enfin, la conception d'une architecture modulaire a demandé un travail rigoureux pour garantir une bonne séparation des responsabilités.

11. Perspectives d'amélioration

Plusieurs évolutions peuvent être envisagées :

- Gestion complète des emprunts et des réservations
- Notifications automatiques
- Application mobile et synchronisation cloud
- Statistiques avancées et recommandations personnalisées par IA
- Reconnaissance vocale

12. Conclusion

Le projet BiblioTech a permis de concevoir une application moderne de gestion de bibliothèque intégrant des technologies actuelles : bases de données relationnelles, interfaces graphiques avancées et intelligence artificielle générative.

L'application répond pleinement aux objectifs du cahier des charges en proposant un système CRUD complet ainsi qu'un chatbot capable de comprendre les questions en langage naturel. Au-delà des aspects techniques, ce projet a permis de développer des compétences

essentielles en conception logicielle, en architecture applicative et en intégration de services d'IA. Il constitue une base solide pouvant évoluer vers une plateforme de gestion documentaire plus complète et plus intelligente.